

AI with Durga Sir
Core Foundations of Generative AI

DURGA
Software Solutions®

Contents

AI 7
Generative AI 7
Google Vs ChatGPT 7
AI 7
- Student 8
- Employee 8
- Business 8

LLM(Large Language Models) 8
How GPT internally works 8
Generative AI Basics 9
- How Transformer works 9
- Process 10

Attention in GPT 10
- Advantages of Attention 11
- Attention concept is required for both input and output tokens 11
- Transformer 11
- How ChatGPT understands the language 11
- Attention mechanism 11

Generating Text 11
How to implement custom tokenizer 11
- Install the tiktoken module 12
- Python Program 12

YouTube videos 12
CPU Vs GPU Vs TPU 13
- QPU 13

Transformer Architecture (Encoder) 14

Input Embeddings 15
Positional Encoding 15
Why it is required 15
Final input to the encoder 15
Multi Head Attention(Self-Attention) 15
Add and Norm 16
Feed Forward 16
Transformer Architecture (Decoder) 17
1. Decoder - Input: 18
2. Output Embedding 18
3. Positional Encoding 18
4. Masked Multi-Head Self Attention 18
5. Add and Norm (After Masked Multi Head Attention) 19
6. Multi Head Attention(cross-Attention) 19
7. Add and Norm (After Multi Head Attention) 20
8. Feed Forward network 20
9. Add and Norm (After Feed Forward) 20
10. Linear 20
11. Softmax 20
For each word, decoder does : 20
Transformers Architecture: MCQs 21
Transformer working understanding: Example-1 24
Step-0: START token 24
Step-1: Masked Multi Head Attention(Look at past only) 25
Step-2: Cross Attention [Look at encoder meaning] 25
Step-3: Feed Forward Network[Extra Thinking] 25
Step-4: Linear scores all words 25
Step-5: Masked Multi Head Attention(2nd round) 25

Step-6: Cross Attention 25
Step-7: Feed Forward 25
Step-8: Linear + Softmax 26
Step-9: Masked Multi Head Attention(3rd round) 26
Step-10: Cross Attention + Feed Forward 26
Step-11: Linear + Softmax 26
Step-12: END Token(stop) 26
What REALLY repeats every time(Key insight) 26
Transformer working understanding : Example-2 27
EX2 — Overall Setup 27
Assume: 27
Final sentence to generate: 27
Important Rule 27
STEP 0 — START Token (Begin signal) 27
Role 27
Human example 27
Decoder has: 27
STEP 1 — Masked Self-Attention (Look at past only) 27
Role 27
Human example 27
Decoder sees: 28
STEP 2 — Cross Attention (Look at encoder meaning) 28
Role 28
Human example 28
Decoder thinks: 28
STEP 3 — Feed Forward Network (Extra thinking) 28
Role 28
Human example 28

STEP 4 — Linear + Softmax (Choose first word) 28
Role 28
Human example 28
First word generated: 28
STEP 5 — Masked Self-Attention (Second round) 29
Role 29
Human example 29
Decoder sees: 29
STEP 6 — Cross Attention (Again) 29
Role 29
Human example 29
Decoder thinks: 29
STEP 7 — Feed Forward Network (Again) 29
Role 29
Human example 29
STEP 8 — Linear + Softmax (Choose second word) 30
Role 30
Human example 30
Second word generated: 30
STEP 9 — Masked Self-Attention (Third round) 30
Role 30
Human example 30
Decoder sees: 30
STEP 10 — Cross Attention + Feed Forward 30
Role 30
Human example 30
STEP 11 — Linear + Softmax (Choose third word) 31
Role 31

Human example 31
Third word generated: 31
STEP 12 — END Token (Stop) 31
Role 31
Human example 31
Decoder generates: 31
What REALLY Repeats Every Time — Key Insight 31
Probability example 33
How GPT remember things 34
Context Window(Working memory - Temporary) 34
Personal Context(Long-term Archive) 34
Hallucination in AI 34
Reasons for Hallucination 35
How to reduce Hallucination 35
Strengths and Limitations of GPTs 35
Strengths 35
Limitations 36
Tokens 37
Why Tokenization is needed in NLP 37
Types of Tokens 39
Token limits 39
Why Token limit is needed 40
How to use GPTs smartly with Token Limits 40
Tokens and Cost(API reality) 41
Why Tokens Cost money 41
How do you control API cost 41
RNN : Recurrent Neural Network 41
LSTM :Long Short Term Memory 42



LSTM has 3 gates(switches) are present 42
Limitations of LSTM 42
Birth of Attention 43
Questions 44
Vocabulary 44
Vector Embeddings 46
Fine Tuning 47
When it is required 47
Process 48
RAG 48
What happens in RAG step-by-step 48



AI

AI ==> Machine Intelligence (Man made thinking power)
- Can take the data
- Can see the data
- Can analyze the data
- Can Identify the patterns
- Decide **and** give the suggestions

Generative AI

Create a new thing ➔ Generate
AI tools generates new content(text, images, audio, video) ➔ Generative AI

suno.ai ➔ audio songs
gamma.ai ➔ PowerPoint presentations
Notebook llm ➔ presentations
- Context-aware learning from your data
- You upload sources (PDFs, docs, notes, links)
- NotebookLM builds a context-grounded understanding
- Answers are based on your material, reducing hallucination
- NotebookLM helps you think **with** your documents

ChatGPT ➔ provide book **and** ask **for** the summary **in** single line **with** 3-5 real examples

Google Vs ChatGPT

Google : Used to find information, **in** simple terms, it **is** a search engine
ChatGPT : Used to get explanations, **in** simple terms, it **is** an answer engine

AI

AI will **not** replace you
A person using AI will replace you
Which skills are needed now?
- Learning mindset
- Using AI tools
- Problem solving skills ➔ Thinking **not** just typing
- Communication ➔ Explaining **and** Speaking
- Adaptability ➔ change **with** time

Student
Learn faster
Practice more
You will shine
Employee
Save time
Do smart work

Business
understand the customers

LLM(Large Language Models)

Brain of AI tool
A LLM **is** an AI System trained on huge data to understand **&** generate human like language

LLM ➔ Brain
Tool ➔ App using that brain

Eg:
ChatGPT **is** a tool using GPT LLM from OpenAI
Gemini **is** a tool using pro/flash LLM from Google
Claude **is** a tool using Sonnet-4.6 LLM from Anthropic
Copilot **is** tool using GPT LLM from Microsoft

How GPT internally works

Chat GPT **is** the Chat AI Application **with** GPT thinking power
Generative Pre-trained Transformer(GPT)
G(Generative) :
It generates content
It creates new sentences.
P(Pre-trained)
It **is** trained before we use it.
Trained on websites, books, Wikipedia, GitHub etc.
Transformer:
This the brain.
Transformer generates data based on pre-trained data

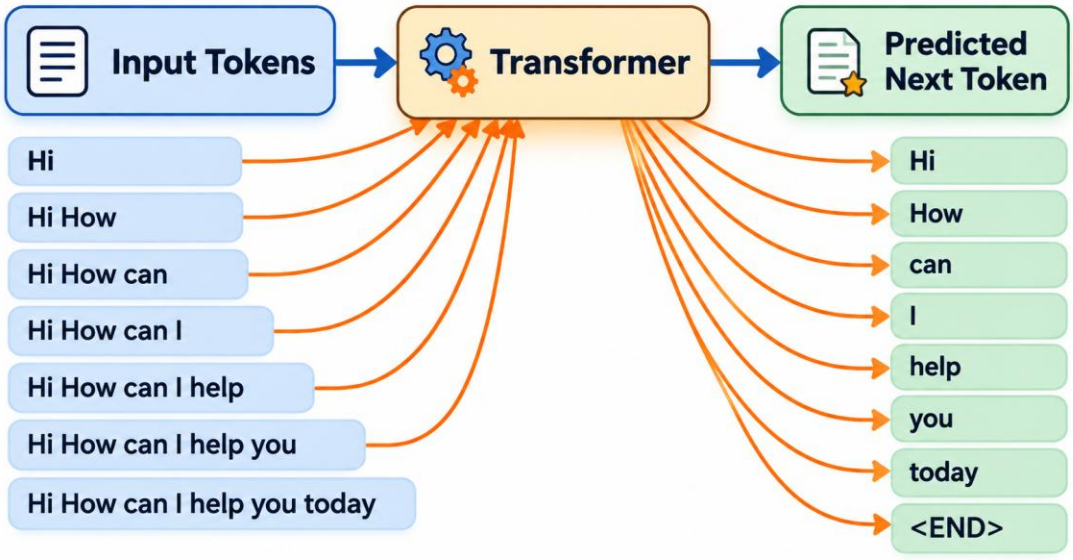
Generative AI Basics

Google people introduced white paper Attention is all you need, which describes GPT architecture

How Transformer works

https://poloclub.github.io/transformer-explainer/

Take input sentence
Convert that sentence into tokens
Convert that token into numbers (Embedding)
Apply "Attention" to find relationships
Predict next word
Return output
At a time only one token **is** predicted.



transformer_working

Process
Prompt ➔ input given to the LLM

For "Hi" it **is** predicted "How"
now the output token "How" **is** added to input token "Hi"
"Hi" **is** the input token **and** earlier generated token **is** "How"
Then the total string ==> "Hi How" **is** considered as input **&** then it **is** generated the next token "can"
So **for** every Iteration the transformer **is** always going to consider "input token" **and** "previously generated output token" to predict the next output

VL-JEPA ==> "Vision-Language Joint Embedding Predictive Architecture"

Attention in GPT
When GPT(Transformer) reads a sentence, it does **not** give equal importance to all words.

It carefully decide which words are important **&** which words are less important **is** there any relation between the words etc.

This thinking process **is** called "Attention"

It will identify that based on pre-trained data **and** based on learning patterns

Input: Ravi did **not pass** the exam because he was sick.
Question: Who was sick? Ravi **or** exam

GPT giving importance **for** Ravi, sick.

Eg: When One person ordered "anna provide Tea with less sugar" **or** another person "give one strong coffee" **or** another person "anna one green tea" to a Tea shop owner.
It will focus on the "tea with less sugar", "strong coffee", "green tea" ➔ this **is** "Attention"

Advantages of Attention
GPT understands context
GPT understands relationships
GPT understands long sentences

Attention concept is required for both input and output tokens
Eg: when teacher asks a question.
You are giving answer, **in** that case you are considering the question **&** answer

Transformer
The smart system that guesses next word by finding patterns.

How ChatGPT understands the language
It uses Transformer model to learn patterns from huge data **&** predict the next best word

https://tiktokenizer.vercel.app/

Attention mechanism
Helps a computer to decide which words are important, focus on them **&** ignore the rest
It helps the model "focus" on important words **and** "ignore" less important ones

Generating Text
Text Generation ➔ Its AI writing **for** you.
AI can write sentences, paragraphs **or** full pages based on your instructions.

How to implement custom tokenizer
Encoding **and** Decoding

"tiktoken" - library used to convert from user input to gpt understandable numeric tokens **(Encoding) and** to convert gpt understandable numeric tokens to human understandable word **(Decoding)**

"pip install tiktoken"

for this model **is** required

Install the tiktoken module

pip install tiktoken

Python Program

```
import tiktoken as tt
encoder = tt.encoding_for_model("gpt-4o")
itext = "Hello ChatGPT how are you"
tokens = encoder.encode(itext)
print(tokens)
```

[13225, 17554, 162016, 1495, 553, 481]

```
import tiktoken as tt
encoder = tt.encoding_for_model("gpt-4o")
itext1 = "Hello ChatGPT how are you"
itext2 = "Hello ChatGPT how are you"
tokens1 = encoder.encode(itext1)
tokens2 = encoder.encode(itext2)
print(tokens1)
print(tokens2)
outtxt = "Hello thanks for asking. what is about you"
tokens = encoder.encode(outtxt)
print(tokens)
```

[13225, 17554, 162016, 1495, 553, 481]
[13225, 17554, 162016, 1495, 553, 481]
[13225, 11707, 395, 16054, 13, 1412, 382, 1078, 481]

```
print(tokens)
text = encoder.decode(tokens)
print(text)
```

[13225, 11707, 395, 16054, 13, 1412, 382, 1078, 481]
Hello thanks for asking. what is about you

YouTube videos

https://youtu.be/X8j_4j50lII ➔ CPU Vs GPU Vs TPU
https://youtu.be/7aS6f_Px3io ➔ Attention



CPU Vs GPU Vs TPU

CPU : Central Processing Unit
- The Generalist Worker
- Like one smart person doing all the work, it's great **for** everyday tasks
- CPUs power our daily digital life
- Speed **for** AI : Slow

GPU : Graphics Processing Unit
- The Parallel Workforce
- Like 100 people working together, its fast **for** bit math **and** AI
- GPUs drive Modern Visuals **and** AI
- Speed **for** AI : Fast

TPU : Tensor Processing Unit
- The AI Factory Machine
- A custom-built specialist, like factory machine designed **for** only one Job:AI
- TPUs run Large-scale AI systems
- Speed **for** AI : Very Fast

Imagine a restaurant that has to prepare 1,000 dishes.

QPU

QPU: Quantum Processing Unit
- Works **with** Quantum bits(qubits)
- Qubits can be 0,1, **or** both at the same time (super position)
- ChatGPT does **not** use QPU
- Used **in** Quantum Research,Drug Discovery, Cryptography research, Physics simulations
- Is still experimental

"CPU" ➔ the main brain of the computer. It's like one smart person doing all t he work, one task at a time.

"GPU" ➔ Has many small workers inside. It's like a team of 100 people working together on a big math problem

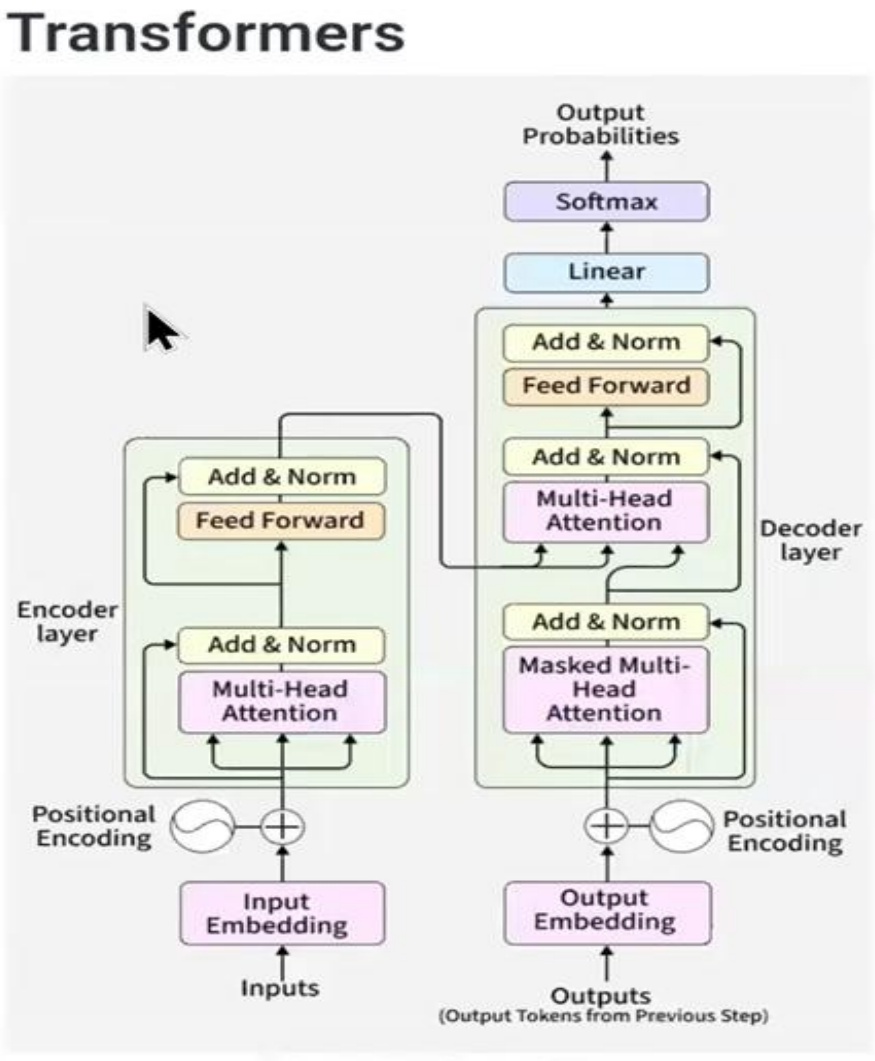
"TPU" ➔ A special machine made only **for** AI work. Like a factory machine built **for** one super-fast job.



Transformer Architecture (Encoder)

Google published a white paper "Attention all you need"
https://arxiv.org/pdf/1706.03762

Reading operation ➔ Encoder
Writing operation ➔ Decoder



Architecture of Transformer

Traansformer Architecture

Input Embeddings
Convert words into numbers, so Transformer understands

Positional Encoding
Transformer don't know about the order
Tell the model the position of each word **in** the sentence to understand the context

Why it is required
Transformer does **not** read the sentence from left to right like humans

Dog bites a man
man bites a man
For Transformer both sentences are same

Final input to the encoder
Final input to the encoder = Input Embeddings + Positional Encoding

Multi Head Attention(Self-Attention)
"Attention"
Transformer focus on most important words **and** relationships between the words to get the main point

Many minds(heads) reading the same input sentence at the same time,but each mind looks **for** a different meaning

Eg: Durga drinks cold mansion house at night

Head1 : Who does the action
"Durga <--> drinks"
This head learns subject/who - action relationship

Head2 : What being consumed
"drinks <--> mansion house"
This head learns action-object relationship

Head3 : Property/ Quality
"cold <--> mansion house"
Learns attribute relation

Head4 : Time

"night <--> drinks"

Learns when this action happens

Head5 : type/category

"mansion house <--> drink type"

This **is** liquid **not** a food

All these heads work **in** parallel to find the meaning **and** relationships
The number of heads **is** transformer design related

Add and Norm

"Add"

We have to add Attention output **with** original input

Add = original input + Attention output

"Normalization"

keep values small **and** balanced

It prevents overflow **and** confusion.

Feed Forward

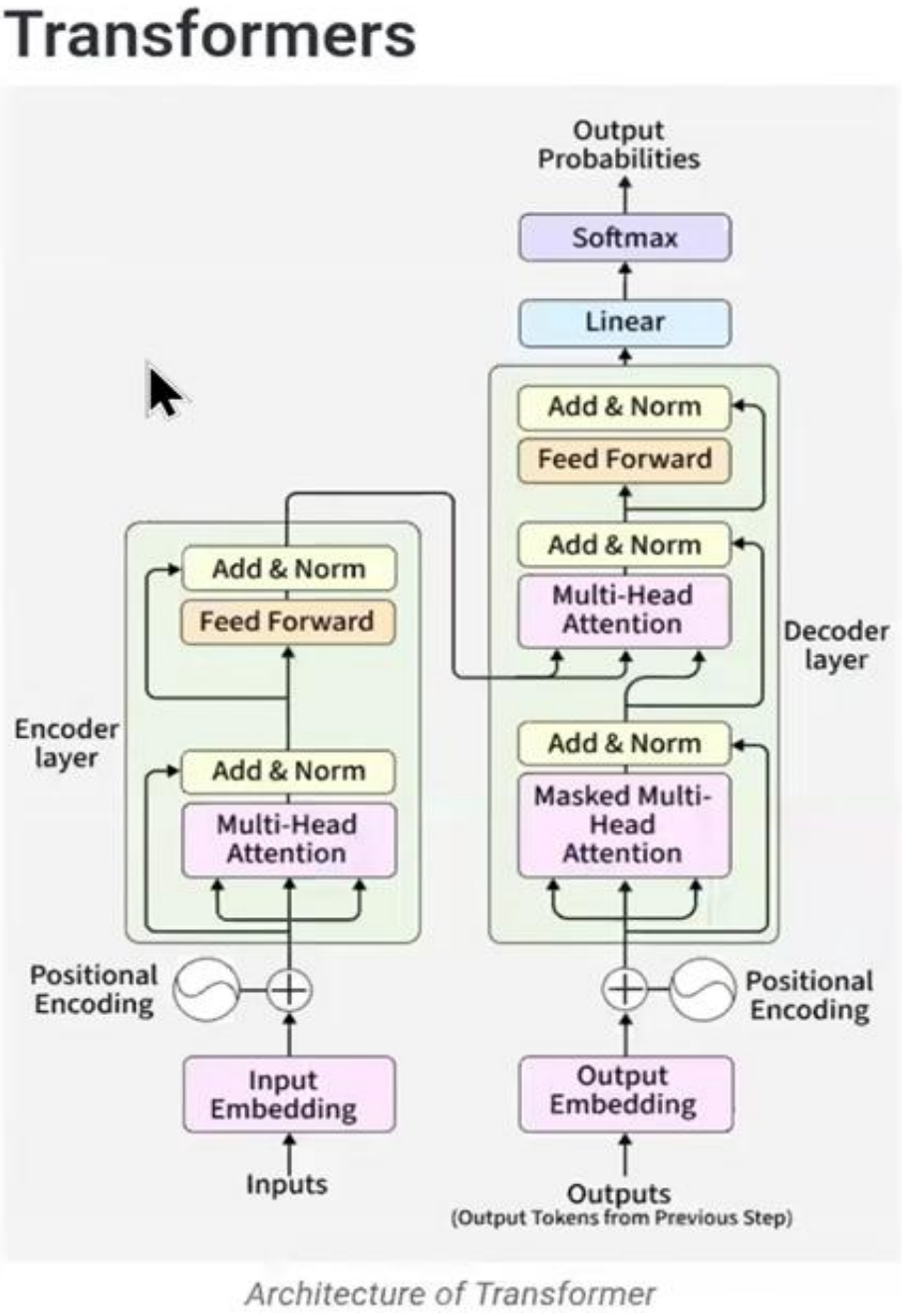
It **is** a deep neural network, it scans every input token **and** trying to add some extra meaningful information about that word.

Each input word **is** individually refined.
Meaning **is** strengthened.
"Durga" ==> understand as a person
"Drinks" ==> understand as action
"cold" ==> understand as quality/condition
"mansion house" ==> understand as a drink item
"at night" ==> understand as time information

Note

"Attention" = Connecting words
"Feed Forward" = Polishing each word

Transformer Architecture (Decoder)



Architecture of Transformer

Transformer Architecture

"Encoder" always talks about "**reading machine**"
"Decoder" always talks about "**writing machine**", predict output **and** provide answers

Encoder can see total input at a time.
Total input **is** available at "Encoder"

Decoder generates only one token at a time

During "AI model training" total input **and** output **is** available
During "Using AI model" **is** always going to predict only one output word at a time.
Word by word generation **is** present

1. Decoder - Input:
Decoder does **not** start from zero.
It takes already generated words.
First time it starts **with** a "START" token

2. Output Embedding
Words are converted into numbers as transformer can understand only numbers

3. Positional Encoding
Position of word **is** added
Without position meaning will be changed

4. Masked Multi-Head Self Attention
Eg:
Q : What **is** my current subject?
Ans : Extreme Ownership
In this scenario we are **not** thinking directly we have answer **is** available

If there **is** no answer **for** the question
Q : What **is** my current subject?
Ans : hide the answer
In this case we are going to think about the question.

Decoder looks only at past words

Future words are hidden(masked)
"This masking concept is applicable at the time of training" where total output statement **is** visible
"Masking is not required at the time of using the model"
Masked attention hides the future words so that model learns to predict, **not** copy.
Prevents cheating.

Masking exists because of "training" **not** because of "usage"
Masking **is not** about input feeding
Masking **is** about attention calculation inside the model
Masking does **not** change the input data
Masking changes how attention **is** calculated

Masking meaning : hiding future words on purpose
- The model **is not** allowed to see future words
- It can see only past words + current word
- This avoids cheating
- Mask = "Don't look ahead" rule
- During Training the model does **not** process one word at a time
- It processes the Full sentence at once(**in** parallel)

- Masking **is not** about input feeding
- Masking **is** about attention calculation inside the model

5. Add and Norm (After Masked Multi Head Attention)
Original input added back
Values are normalized
Keep learning stable
Avoids confusion

6. Multi Head Attention(cross-Attention)
Encoder - Decoder Attention (Cross Attention)
Decoder looks at Encoder output
Connects input meaning **with** output generation
Helps generate correct **and** relevant words

7. Add and Norm (After Multi Head Attention)

Original input added back
Values are normalized
Keep learning stable
Avoids confusion

8. Feed Forward network

Extra thinking layer
improves understanding quality
Helps grammar **and** structure

9. Add and Norm (After Feed Forward)

Original input added back
Values are normalized
Keep learning stable
Avoids confusion

10. Linear

This **is** last thinking step
It converts model thinking into word scores
It does **not** choose the word
It only prepare options

11. Softmax

Converts scores into probabilities
Choose the best next word
Highest probability word will be selected

For each word, decoder does :

1. Look at past words(Masked Multi Head Attention)
2. Look at input meaning (Cross Attention/ Encoder-Decoder attention)
3. Think deeper (Feed Forward)
4. Score all words (Linear Layer)
5. Pick One word (Softmax Layer)



Transformers Architecture: MCQs

Q1. In Transformer diagram, what **is** the main job of Encoder?
A. Generate output words
B. Translate language
C. Understand input sentence
D. Store data

Ans : C

Q2. Which part generates output word by word
A. Encoder
B. Decoder
C. Embedding
D. Tokenizer

Ans: B

Q3. What does "Masked self-attention" mean in Decoder
A. Hides input sentence
B. Sees only future words
C. Sees only past generated words
D. Removes attention

Ans: C

Q4. Why **is** attention **is** used **in** Transformer
A. To increase memory size
B. To focus on important words
C. To reduce speed
D. To remove grammar

Ans: B

Q5. What **is** the role of Softmax **in** the diagram
A. Converts words to numbers
B. Adds position information
C. Choose next best word
D. Stops the model

Ans: C

Q6. Where **is** "Self-attention" used
A. Only **in** Encoder
B. Only in Decoder
C. In both Encoder **and** Decoder
D. Not used

Ans: C

Q7. Encoder output **is** mainly used by which part
A. Tokenizer
B. Input layer
C. Decoder
D. Softmax

Ans: C

Q8. Transformer architecture mainly works based on which concept
A. Memory
B. Attention
C. Storage
D. Compression

Ans: B

Q9. Which part of Transformer understands the input sentence
A. Decoder
B. Softmax
C. Encoder
D. Tokenizer

Ans: C

Q10. Which part generates the output word by word
A. Encoder
B. Decoder
C. Embedding
D. Position Encoding

Ans: B

Q11. What does Tokenization mean
A. Removing words
B. Translating language
C. Breaking sentence into small parts
D. Encrypting text

Ans: C

Q12. Why **is** "Positional Encoding needed"
A. To add color
B. To store data
C. To know word order
D. To reduce size

Ans: C

Q13. What **is** Self-attention
A. Ignoring words
B. Focusing on one word only
C. Each word looking at all other words
D. Removing grammar

Ans: C

Q14. What **is** Masked Self Attention used **for**
A. To hide input
B. To block encoder
C. To prevent seeing future words
D. To remove output

Ans: C

Q15. Multi Head Attention means
A. One attention only
B. Multiple attentions working together
C. No attention
D. Random attention

Ans: B

Q16. What **is** the role of "Softmax"
A. Converts text to image
B. Adds position
C. Choose next word using probability
D. Stops training

Ans: C

Q17. Which models are replaced by Transformers
A. CNN only
B. RNN **and** LSTM
C. Database models
D. Rule-based systems

Ans: B

Transformer working understanding: Example-1
Encoder input understanding: "What is today's weather"
Decoder output: Rain **is** heavy

Step-0: START token
Tell the decoder ==> "start generating output"

You open the mouth to speak
No word spoken yet
Decoder has [START]

Step-1: Masked Multi Head Attention(Look at past only)
Masked Multi Head Attention look only past words
No words generated yet
Decoder sees only [STRAT]

Step-2: Cross Attention [Look at encoder meaning]
You remember the question ==> "What is today's weather"
Decoder thinks
I must talk about weather

Step-3: Feed Forward Network[Extra Thinking]
Decoder thinks
weather -- rain. may be start **with** rain

Step-4: Linear scores all words
Rain
Sun
Wind
Softmax picks the best word : "rain" (first word generated)

Step-5: Masked Multi Head Attention(2nd round)
Already said : Rain

Decoder Thinks : Sentence **is not** complete
Decoder sees : Rain

Step-6: Cross Attention
Rechecks input meaning
Avoid going off topic

Question remainder : Describes todays weather?
Decoder Thinks : I should describe rain

Step-7: Feed Forward
Extra thinking layer
Describes grammar pattern **and** structure?
Rain .....**is**.....something

Step-8: Linear + Softmax

Options
- **is**
- was
- falls

Softmax selected ➔"is"
Second word generated ➔ "is"

Positional Encoding preserves the order **and** position of words

Step-9: Masked Multi Head Attention(3rd round)

Keep track of sentence so far
Decoder sees ➔ "Rain is"
Listener expects more

Step-10: Cross Attention + Feed Forward

Decide description word
Keep answer aligned to input
Decoder think : "How is rain"

Step-11: Linear + Softmax

Options
- heavy
- sweet
- fast

Third word generated **is** "Heavy"
"Rain is heavy"

Step-12: END Token(stop)

Signal sentence completion
Human : Thought **is** complete. You stop speaking

What REALLY repeats every time(Key insight)

For each word, decoder repeats
- Look at past words ➔ "Masked Self Attention"
- Look at input meaning ➔ "Cross Attention"
- Think deeper ➔ "Feed Forward"
- Prepare options ➔ "Linear"
- Choose Word ➔ "Softmax"

Transformer working understanding : Example-2

EX2 — Overall Setup

Assume:
Encoder already understood the input:
"Tell me about the shop status"

Final sentence to generate:
"Shop is closed"

Now ONLY DECODER **is** working

Important Rule
The decoder does NOT know the full sentence.
It creates the sentence one word at a time

STEP 0 — START Token (Begin signal)

Role
Tells decoder to start generating output.

Human example
You are about to speak.
Mouth opens, no words yet.

Decoder has:
START

STEP 1 — Masked Self-Attention (Look at past only)

Role
Looks only at what **is** already written.
Future words are hidden.
Maintains correct word order.

Human example
You have said nothing yet.
You think:
"How do I begin?"

Decoder sees:
START

STEP 2 — Cross Attention (Look at encoder meaning)

Role
Reminds decoder what the topic **is**.
Keeps the answer relevant.

Human example
You remember the question:
"Tell me about the shop status"

Decoder thinks:
"I must talk about the shop"

STEP 3 — Feed Forward Network (Extra thinking)

Role
Decides grammar pattern.
Decides what type of word **is** needed.

Human example
You think:
"Sentence should start with Shop"

STEP 4 — Linear + Softmax (Choose first word)

Role
Linear: gives scores to all words.
Softmax: selects the best word.

Human example
Options **in** mind:
Shop ✅
Market ❌
Road ❌

First word generated:
Shop

STEP 5 — Masked Self-Attention (Second round)

Role
Looks at words written so far.
Maintains sentence flow.

Human example
You already said:
"Shop"
You think:
"Sentence is incomplete"

Decoder sees:
Shop

STEP 6 — Cross Attention (Again)

Role
Re-checks input meaning.
Avoids going off-topic.

Human example
Question reminder:
"Tell me about the shop status"

Decoder thinks:
"I should describe shop condition"

STEP 7 — Feed Forward Network (Again)

Role
Decides the next grammar word.
Verb comes next.

Human example
You think:
"Shop ... is ..."

STEP 8 — Linear + Softmax (Choose second word)

Role
Scores words.
Picks the best fit.

Human example
Options:

- **is** ✅
- was ❌
- runs ❌

Second word generated:
Shop **is**

STEP 9 — Masked Self-Attention (Third round)

Role
Tracks the sentence so far.

Human example
You said:

"Shop is"

The listener **is** waiting.

Decoder sees:
Shop **is**

STEP 10 — Cross Attention + Feed Forward

Role
Decides the description word.
Matches it **with** the input meaning.

Human example
You think:

"What is the shop status?"

The decoder now understands that it needs a word describing the shop's condition.

STEP 11 — Linear + Softmax (Choose third word)

Role
Final word decision.

Human example
Options:
- closed ✅
- sweet ❌
- fast ❌

Third word generated:
Shop **is** closed

STEP 12 — END Token (Stop)

Role
Signals sentence completion.

Human example
Thought **is** complete.
You stop speaking.

Decoder generates:
END

Therefore, the final output **is**:
"Shop is closed"

What REALLY Repeats Every Time — Key Insight

For each new word, the decoder repeats:

1. Look at past words → Masked Self-Attention
2. Look at input meaning → Cross Attention
3. Think deeper → Feed Forward

4. Prepare options / scores → Linear
5. Choose the next word → Softmax

ML Example

pip install scikit-learn
sklearn ==> very popular python library **for** ML
Model
  Train this model **with** my input data
  predict my required output

model.fit(hours, marks)
  model looks at hours **and** marks
  model finds relationship
  It will create some equation like ==> "marks = m*hours + b "
  "m" slope(how fast marks increase) ==> "model.coef_"
  "b" = base marks(standard value) ==> "model.intercept_"
model.score(hours,marks) ==> gives the score

```
from sklearn.linear_model import LinearRegression
model = LinearRegression() # we are creating an empty brain
hours = [[1],[2],[3],[4],[5]]
marks = [35,40,50,60,75]
model.fit(hours, marks)  # Learn from data
model.predict([[6]])
```

array([82.])

```
model.coef_
```

array([10.])

```
model.intercept_
```

np.float64(21.999999999999993)

```
model.coef_ * 6 + model.intercept_
```

array([82.])

Note

marks = m*hours + b
m(slope) = model.coef_
b(base marks) = model.coef_
marks = model.coef_ * 6 + model.intercept_

```
model.score(hours,marks)
```

0.970873786407767

```
from sklearn.linear_model import LinearRegression
model = LinearRegression() # we are creating an empty brain
experience = [[0],[1],[2],[3],[4],[5]]
salary = [10,15,22,30,38,50]
model.fit(experience, salary)  # learn from data
model.predict([[6]])
```

array([55.2])

```
print("The predicted salary of 10 years of exp :",model.predict([[10]]))
```

The predicted salary of 10 years of exp : [86.85714286]

```
model.score(experience,salary)
```

0.9826343087660883

Probability example

```
import random
words = ['morning','evening','night']
probability = [0.6,0.3,0.1]
print("Usually I am taking tea at :", random.choices(words,probability))
```

Usually I am taking tea at : ['evening']

```
print("Usually I am taking tea at :", random.choices(words,probability))
```

Usually I am taking tea at : ['morning']

```
print("Usually I am taking tea at :", random.choices(words,probability))
```

Usually I am taking tea at : ['night']

```
help(random.choices)
```

Help on method choices in module random:

choices(population, weights=None, *, cum_weights=None, k=1) method of random.Random instance
    Return a k sized list of population elements chosen with replacement.

    If the relative weights or cumulative weights are not specified,
    the selections are made with equal probability.

How GPT remember things

GPTs works **with** 2 types of memory
  Context Window(Working memory - Temporary)
  Personal Context(Long-term Archive)

Context Window(Working memory - Temporary)

It remembers what we talk inside one chat
When the chat ends it **is** cleared
Capacity **is** limited. If this chat becomes 500 pages long, it will start forgetting the very first messages

Personal Context(Long-term Archive)

This **is** long term memory
It remembers important user details like
  name, learning style, preferences
It works across many chats

"Context Window" ➔ Remembers the current chat conversation
"Personal Context" ➔ Remembers the person across multiple chats

Hallucination in AI

AI gives an answer that looks correct, sounds confident, but it **is** actually wrong **or not** verified.
This **is** called "Hallucination"

Reasons for Hallucination

1. GPT predicts, **not** checks
2. Missing **or** confusing data
   model may **not** have : exact latest information
   model may **not** have : exact local information
3. Your prompt **is** vague

How to reduce Hallucination

1. Ask **for** sources
   Say: Give sources **or** links
2. Ask it to say I don't know
   Say : If you are **not** sure, say you are **not** sure
3. Be specific
   Vague : Tell me about Java
   Specific : Explain java 21 virtual threads **in** simple words **with** one example

Strengths and Limitations of GPTs

Strengths

1.Explaining concepts clearly
   Explaining basics
   Breaking complex topics
   Give examples

2.Writing **and** Drafting content
  GPT can
   Write emails
   Create resumes
   Draft Documents
   Prepare notes

3.Coding Help
  GPT can
   Explain the code
   Generate the code
   Debug simple issues

4.Language **and** communications
Grammer correction
Sentence rewriting
Translation
Summarization

5.Brainstroming **and** Ideas
GPT **is** good **for**
Project ideas
Startup ideas
Learning paths
Topic Suggestions

Limitations

1.No real understanding **or** emotions
GPT cannot
think
feel
understand like human
It just predict words that's all

2.Can be Confidently wrong
Hallucination
Never trust blindly **for** facts

3.No Real time awareness
GPT may **not**
Know latest news
Know live prices
Know today's events
unless connected to tools

4.No responsibility
If GPT give wrong advice
GPT **is not** responsible
You are responsible

5.So never use GPT **for**
Medical Decisions
Legal Decisions
Financial Investment decisions without expert verification

if task **is** learning **or** drafting:
use GPT confidently
elif task **is** critical decision:
verify **with** experts
else:
use GPT as assistant, **not** authority

Tokens

Smallest unit of Text that AI understands
A word can be a token **or** many tokens

Why Tokenization is needed in NLP

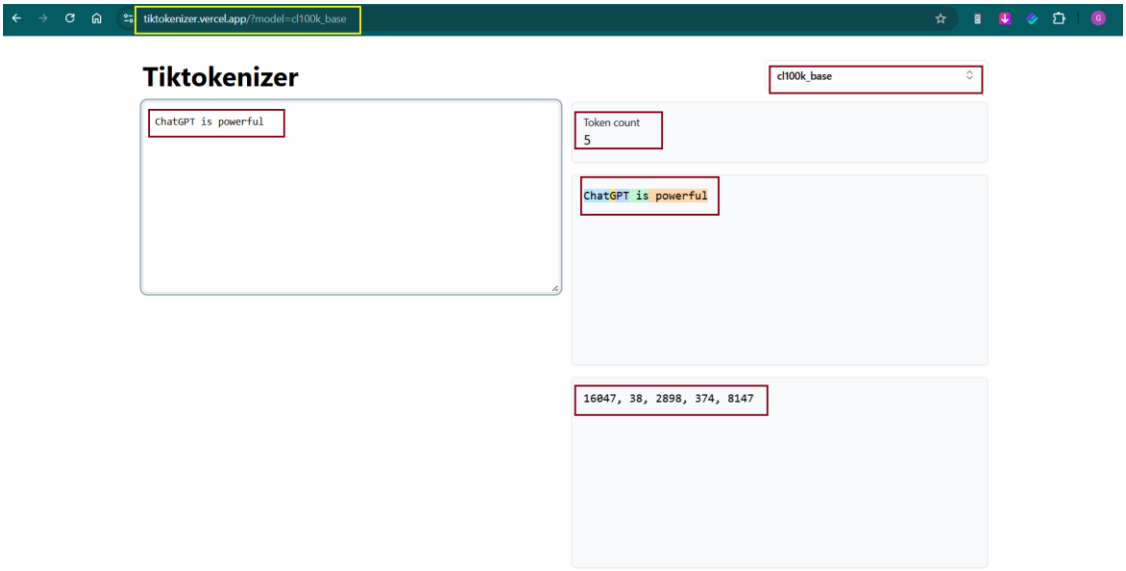
Tokenization **is** needed because computers cannot understand raw text
Tokenization breaks text into small units, which can be converted into numbers **for** processing by NLP models

"https://tiktokenizer.vercel.app/?model=cl100k_base"

```
import tiktoken as tt
enc = tt.get_encoding("cl100k_base")
text ="ChatGPT is powerful"
tokens = enc.encode(text)
print(tokens)
print(len(tokens))
decoded = enc.decode(tokens) # tokens to text
print(decoded)
```

[16047, 38, 2898, 374, 8147]
5
ChatGPT is powerful

https://tiktokenizer.vercel.app/?model=cl100k_base

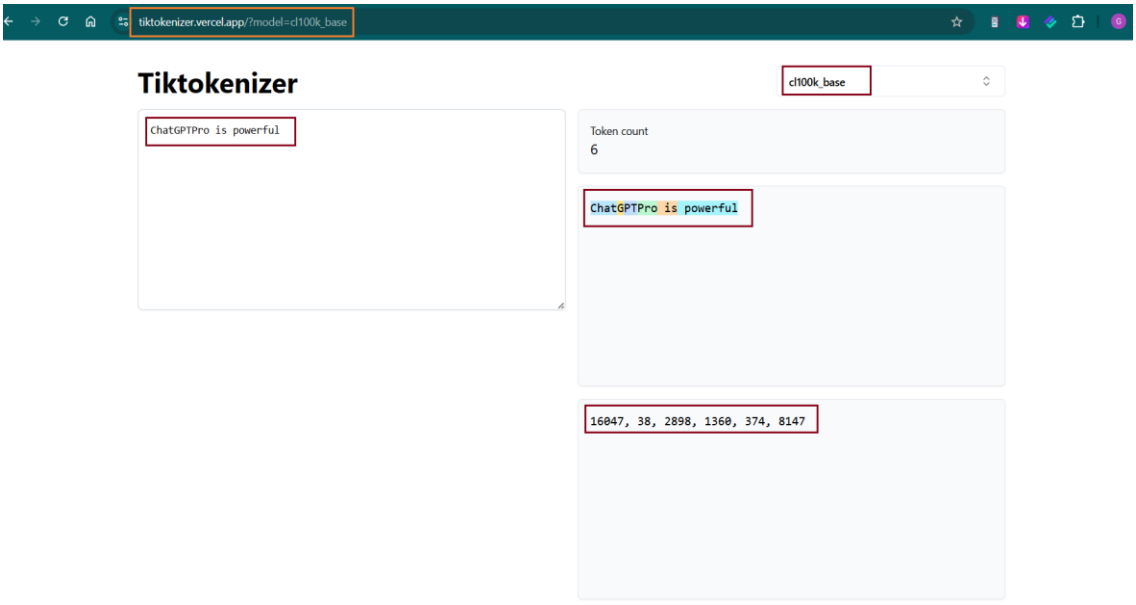


tiktokenizer

```
import tiktoken as tt
enc = tt.get_encoding("cl100k_base")
text ="ChatGPTPro is powerful"
tokens = enc.encode(text)
print(tokens)
print(len(tokens))
decoded = enc.decode(tokens) # tokens to text
print(decoded)
```

[16047, 38, 2898, 1360, 374, 8147]
6
ChatGPTPro is powerful

https://tiktokenizer.vercel.app/?model=cl100k_base



tiktokenizer

Types of Tokens

1. Word Tokens (Old GPTs) ➔ Every word **is** treated as separate token
   Simple NLP Tasks
2. Subword Tokens (new GPTs) ➔ Words are broken into meaningful smaller parts
   GPTs/LLMs
3. Character Tokens ==> Each character **or** symbol **is** one token
   Special Research

Token limits

The maximum number of tokens GPT can read(input) **and** write(output) **in** one interaction

"Token limit = input tokens + output tokens"

Why Token limit is needed

1. Memory **and** Performance
   More Tokens means
      More memory usage
      More Computation
      Slower Response
   Token limit keep GPT :
      Fast
      Stable
      Efficient

2. Context Window size
   If the input **is** too big:
     Old tokens are dropped
     confusion happens

3. Cost Control
   More Tokens = More Cost

4. Quality of output
   Very long input can :
   Confuse the model
   Reduce Focus
   Reduce answer quality

How to use GPTs smartly with Token Limits

1. Be clear **and** short
   Avoid unnecessary words
   Ask precise questions

2. Break big tasks
   instead of one long prompt, use multiple small prompts

3. Control output size
   Ask:
     Explain **in** 5 points
     Answer **in** 100 Words

This save Tokens

Tokens and Cost(API reality)

Tea ➔ Paying based on cups
Data ➔ Paying based on GBs
Electricity ➔ Paying based on units
AI ➔ Paying based on Tokens

Input tokens **and** output tokens both cost money

Why Tokens Cost money

Behind AI:
- Huge servers are running
- Powerful GPUs are working
- Electricity **is** consumed
- Engineers maintain systems

AI **is not** magic. AI **is** heavy computation

Every Token means
- Calculation
- Memory Usage
- Processing time

Companies are charging based on Tokens

How do you control API cost

By reducing Token usage
By limiting prompt size
By limiting output length
By caching responses

RNN : Recurrent Neural Network

Yesterday night I had fever,
Today morning headache
Afternoon stomach pain
and now body pain

It will read the text word by word **and** unable to read complete input at a time
The processing will become slow

As memory **is** limited, **for** long input, old stored word will be override **with** new words

There **is** no mechanism to check whether the given input word **is** important **or not**
Hence it will store all words

To overcome these problems next level we used LSTM

LSTM :Long Short Term Memory
LSTM = Long + Short + Memory

Current words are stored **in** "Short Memory"
The words which are important will be stored **in** "Long Memory"
In Short memory overriding may be possible

LSTM has 3 gates(switches) are present
"Forget Gate" : "Remove useless past"
"Input Gate" : "Store important information"
"Output Gate" : "Decide what to speak"

Limitations of LSTM
1. It will read text word by word **and** unable to read complete input at a time.
The processing will become slow

2. Even though more memory when compared **with** RNN, Still it unable to process very long input sentences.

3. To overcome these problems next level we have to go **for** "Transformers"

Birth of Attention

These limitations created questions:

1. Why **not** look at all words at the same time?
2. Why **not** vie importance to relevant words?

To answer these questions, "Attention" came where

Focus on important words
Ignore less important words
Process total input at a time

Feature	RNN	LSTM	Transformer
Full form	Recurrent Neural Network	Long Short-Term Memory	Transformer Model
Memory type	One small memory	Long + short memory	No single memory
Remembers long past?	✗ No	✓ Better	✓✓ Excellent
Forgetting problem	✗ High	◆ Reduced	✗ None
How it reads sentence	One word at a time	One word at a time	All words together
Speed	✗ Slow	✗ Slow	✓ Very fast
Parallel processing	✗ No	✗ No	✓ Yes
Handles long sentences	✗ Poor	◆ Medium	✓ Excellent
Key idea	Pass memory step by step	Memory with gates	Attention on all words
Main weakness	Forgets old info	Still slow	Needs more compute
Used today in ChatGPT?	✗ No	✗ No	✓ Yes

RNN_LSTM_TRANSFORMER

Questions

Q: Why RNN/LSTM failed **for** long sentences

Ans:
RNN **and** LSTM process text sequentially **and** struggled to retain long-range de pendencies, causing them to forget earlier important words **in** long sentences.

Word by word processing
Limited memory

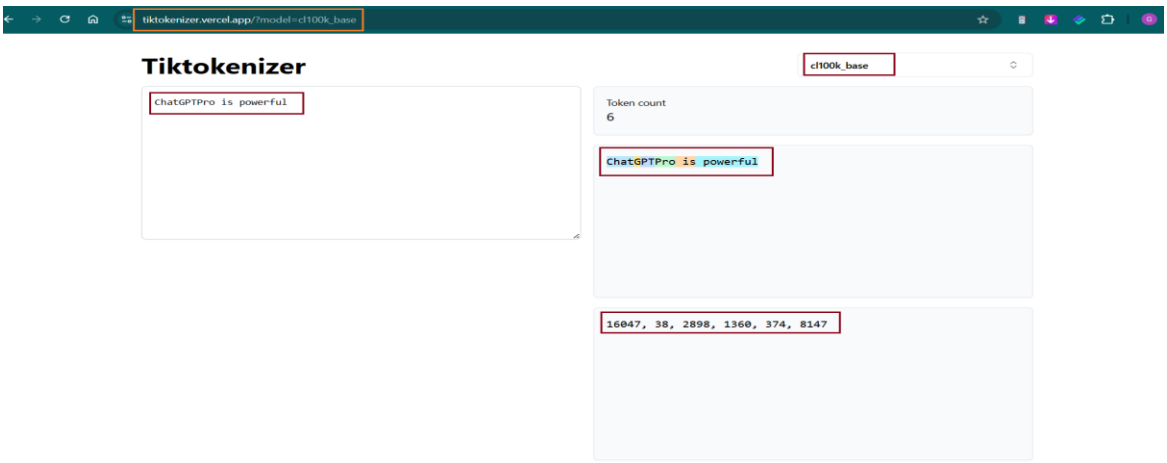
https://projector.tensorflow.org/

Vocabulary

Every model **is** having internally vocabulary.
Based on the vocabulary it will search

"ChatGPTPro" ➔ this word will **not** be there **in** the vocabulary
It will search **for** the sub words `Chat`, `G` ,`PT` **and** `Pro`
which are present **in** the vocabulary
Like this any new word will be searched

https://tiktokenizer.vercel.app/?model=cl100k_base



tiktokenizer

word based token

Tokenizer breaks the sentences into tokens
Each token already exists **in** vocabulary(vocab) **and** has a fixed token Id
Id **is** something like index

Vocabulary **is** fixed list of all tokens that tokenizer already knows
Created before the model released.
Same **for** all users
Never changes during usage
Shared between tokenizer **and** embedding layer
Vocabulary does **not** contain
- Meanings
- Grammer rules
- Intelligence

Vocabulary contains only
- Token text
- Token ID(number like index)

Vocabulary **is** the read-only look-up table.

Input:
I want to learn AI

I ➔ 101
want ➔ 2345
learn ➔ 7895
AI ➔ 9901

101, 2345, 7895, 9901 are the Token Ids coming from Vocabulary.

Token Ids ==> will be converted into vector embeddings

Vector Embeddings

Token Ids ==> will be converted into vector embeddings
Token ==> Token ID ==> Vector Embeddings
Apple ==> 101(Token Id) ==> [1.0,2.5,3.6,.....](Vector Embeddings)

0 to 10

Representing Objects as Feature Vectors

Object	Is it Food?	Is it a Machine?	Is it Round?	Does it have Wheels?	Does it Grow?
Apple	10	0	8	0	10
Orange	10	0	10	0	10
Car	0	10	2	10	0
Bike	0	8	4	10	0

Each object is represented as a numerical feature vector based on its properties.

embeddings

Object	Feature Vector
Apple	[10, 0, 8, 0, 10]
Orange	[10, 0, 10, 0, 10]
Car	[0, 10, 2, 10, 0]
Bike	[0, 8, 4, 10, 0]

The machine calculates the distance between the
Apple [10,0,8,0,10] **and** Orange [10,0,10,0,10].

It **is** very small **and** hence machine identified these are nearly identical

Fine Tuning

Fine Tuning means training already trained AI Model again using our own exa mple data to change its behavior **and** response style

Eg: A General MBBS person after fine tuning becomes specialist

When it is required

If you want same tone every time
If you want fixed format
If you want domain specific answers

Eg1: Customer Support bots
- Same tone
- Same type of replies
- Company approved language

Eg2: Medical/Legal Drafting
- Fixed format
- No creativity
- High consistency

Q: Already prompt **is** there, what **is** the need of finetuning?

Ans:
Same question
Same prompt
But AI gives different style answers every time

Finetuning makes AI consistent

Note:
Finetuning mainly meant **for** behavior change

Process

1. Start **with** already trained model
2. Prepare your own example data(custom data)
3. Show these examples to the model many times
4. Model slowly adjusts itself
5. A new specialized model created

RAG

Retrieval Augmented Generation
User asks question it first goes to Vector Databases
"Retrieving" the data from Vector databases
The response(top k results) are "augmented(added)" to the Query
Now both the Query **and** Augmented results are given to the LLM
LLM will "generate" the response

What happens in RAG step-by-step

1. User asks a question
2. Question **is** converted into external embeddings
3. Vector DB finds relevant document chunks
4. Retrieved text + question **is** sent to LLM
5. LLM uses internal embeddings + attention
6. Final answer **is** generated

Feature	Fine-Tuning	RAG (Retrieval-Augmented Generation)
Basic idea	Train the model again on your data	Fetch relevant data first, then ask the model
Model change	✅ Model weights are changed	❌ Model weights are **not** changed
Data usage	Data influences what the model learns	Data is retrieved and **referenced** at query time
Where data is stored	Knowledge is encoded in model parameters	Outside the model, typically in a **Vector Database**

Embeddings used	Model's existing embedding/representation layers are part of the model	External embeddings for retrieval + model's internal representations
Best for	Fixed style, tone, format, behavior, or domain-specific patterns	Latest facts, large documents, FAQs, policies, manuals
Update data	Hard — retraining/fine-tuning is needed	Easy — update the documents/index
Hallucination risk	Can still hallucinate, especially when information is outdated or absent	Can be reduced by grounding responses in retrieved documents
Real-time data	❌ Not suitable by itself	✅ Suitable when connected to up-to-date data sources
Cost	Higher — requires training/fine-tuning	Generally lower — no model retraining required
Speed	Fast at runtime because there is no retrieval step	Slightly slower because of the retrieval step
Control over content	Less direct control over individual facts	More control through retrieved documents and sources
Example use	Chatbot tone, writing style, response format	Company FAQ, policies, manuals, knowledge bases
Scales with data	❌ More difficult as the knowledge base continually changes	✅ Easier — documents can be added or updated
Knowledge freshness	Depends on when the model was fine-tuned	Can be kept current by updating the knowledge source
Best suited for	**Changing how the model behaves**	**Giving the model access to changing information**

Fine-tuning teaches the model
RAG lets the model refer notes before answering

"Fine-Tuning" → Teaching a student **and** expecting him to remember forever
"RAG" → Student opens notes before answering exam

Static Data ==> Fine-tuning **is** best suitable
Dynamic Data ==> RAG **is** best suitable